

SECTION 2 — OBJECT-ORIENTED PROGRAMMING (Senior SDET Masterclass)

Topics Covered

- 2.1 Encapsulation (Information Hiding & Invariant Enforcement in POM)
 - 2.2 Inheritance (Hierarchical Boundaries & Fragile Base Class Anti-Pattern)
 - 2.3 Polymorphism (Dynamic vs Static Polymorphism in Multi-Browser Drivers)
 - 2.4 Abstraction (Contract-Driven Test Layers & Core Decoupling)
 - 2.5 Composition vs. Inheritance (Favoring 'Has-A' over 'Is-A' in Framework Design)
 - 2.6 Association, Aggregation, and Composition (UML Lifecycle Coupling in Test Components)
 - 2.7 Interface vs. Abstract Class (Architectural Decision Framework)
 - 2.8 Default and Static Interface Methods (Java 8+ Evolution & Diamond Resolution)
 - 2.9 Method Overloading (Compile-Time Resolution & Type Promotion Rules)
 - 2.10 Method Overriding (Runtime Dispatch & Covariant Return Types)
 - 2.11 Dynamic Method Dispatch (JVM Virtual Method Table / vtable Internals)
 - 2.12 Constructor Chaining (`this()` , `super()` , & Object Initialization Sequence)
 - 2.13 `this` vs. `super` (Scope Resolution, Shadowing, and Reference Escaping)
 - 2.14 Access Modifiers (Encapsulation Boundaries & Modular Architecture)
 - 2.15 `static` Mechanics (Classloaders, Thread Safety Traps, & Memory Leaks)
 - 2.16 `final` Mechanics (Immutability, JMM Memory Barriers, & Inline Optimization)
 - 2.17 SOLID Principles (Deep Theoretical Mechanics)
 - 2.18 SOLID Applied to Test Automation (5,000+ Test Suite Scalability)
 - 2.19 High-Stakes Senior OOP Interview Questions & Whiteboard Scripts
-

2.1 Encapsulation

1. Theory & Core Mechanics

Encapsulation is the bundling of data (fields) and the behaviors (methods) operating on that data into a cohesive unit, while strictly hiding internal implementation details behind private access modifiers and public intent-revealing contracts.

- **State Invariant Protection:** Fields cannot be placed into invalid or corrupt states from external callers. The class constructor and mutator methods enforce valid boundaries.
 - **Decoupled Evolution:** The internal implementation (e.g., swapping a Selenium `By.xpath` with `By.cssSelector` , or changing an in-memory `List` to a `Set`) can be modified without breaking a single external caller.
-

2. Enterprise Relevance (5,000+ Test Scale)

In large test frameworks spanning multiple agile squads, encapsulation prevents tests from directly manipulating browser drivers, raw locator strings, or low-level wait loops. If an application undergoes a major front-end redesign

(e.g. migrating from AngularJS to React with new DOM attributes), encapsulation guarantees you update **one Page Object class**, rather than fixing 400 broken test methods scattered across 50 repository branches.

3. Production-Ready Code: Encapsulated Page Object

```
package com.deloitte.sdet.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.TimeoutException;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;
import java.util.Objects;

/**
 * Encapsulated Page Object:
 * - All WebElements and By locators are strictly private.
 * - Public API exposes user business intent, never raw Selenium commands.
 * - Invariants and wait boundaries are enforced internally.
 */
public final class SecureCheckoutPage {

    private final WebDriver driver;
    private final WebDriverWait wait;

    // Strict encapsulation: Locators are completely invisible to tests
    private final By cardNumberInput = By.id("cc-number");
    private final By expirationInput = By.id("cc-exp");
    private final By cvvInput = By.id("cc-cvv");
    private final By payButton = By.cssSelector("[data-testid='pay-submit-btn']");
    private final By successAlert = By.cssSelector(".alert-success");

    public SecureCheckoutPage(WebDriver driver) {
        this.driver = Objects.requireNonNull(driver, "WebDriver cannot be null");
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    /**
     * Public business workflow encapsulating sequencing, clearing, typing, and synchronization.
     */
    public boolean submitPayment(String cardNum, String exp, String cvv) {
        try {
            wait.until(ExpectedConditions.visibilityOfElementLocated(cardNumberInput)).sendKeys(
                driver.findElement(expirationInput).sendKeys(exp);
                driver.findElement(cvvInput).sendKeys(cvv);

            wait.until(ExpectedConditions.elementToBeClickable(payButton)).click();

            return wait.until(ExpectedConditions.visibilityOfElementLocated(successAlert)).isDis
        } catch (TimeoutException e) {
            throw new RuntimeException("Payment submission timed out at URL: " + driver.getCurre
        }
    }
}
```

```
}
}
```

4. High-Stakes Scenario: UI Component Rework Breaks Hundreds of Tests

- **Context:** A developer changes the `#submit-order` button from a standard `<button>` to a custom Shadow-DOM web component `<order-button>`.
- **Triage:**
 - **Unencapsulated suite:** Tests contained raw `driver.findElement(By.id("submit-order"))`. 320 tests fail simultaneously in CI. Triage requires updating 320 files across 6 teams, taking 3 days.
 - **Encapsulated suite:** Tests only invoked `checkoutPage.submitOrder()`. The change is resolved by updating only `CheckoutPage.java` using a JavaScript executor or Shadow-DOM locator. Time to fix: 15 minutes; zero tests modified.

5. Anti-Patterns & Pitfalls

- **✗ Declaring `public WebElement submitBtn` in Page Objects:** Exposes the internal DOM node to the test layer, inviting tests to invoke raw `.click()` without synchronization.
- **✗ Exposing public setters for private fields:** Allows tests to inject unvalidated state into Page Objects, destroying invariants.

2.2 Inheritance

1. Theory & Core Mechanics

Inheritance represents an **"Is-A" relationship**, allowing a derived class (subclass) to inherit state (fields) and behavior (methods) from a base class (superclass).

- Java supports **single inheritance for classes** (a class can only extend one superclass) rooted at `java.lang.Object`.
- Constructors are **not inherited**; a subclass constructor must invoke a superclass constructor via `super()` as its first statement.
- `private` members of the superclass exist in the subclass instance memory, but are not directly accessible.

2. Enterprise Relevance & The "Fragile Base Class" Problem

While inheritance is useful for providing common setup in `BaseTest` or `BasePage`, deep inheritance hierarchies (e.g. `BaseComponent` extends `BasePage` extends `BaseUI` extends `DriverInit`) create severe architectural fragility:

- Any modification to `BasePage` (such as changing default wait times or adding a logging interceptor) unpredictably cascades to every subclass.
- Subclasses inherit methods that may make no conceptual sense for them.

3. Production-Ready Code: Controlled Inheritance Hierarchy

```

package com.deloitte.sdet.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;
import java.util.Objects;

/**
 * BasePage provides reusable primitives for DOM synchronization and interaction.
 * Marked abstract to prevent direct instantiation.
 */
public abstract class BasePage {

    protected final WebDriver driver;
    protected final WebDriverWait wait;

    protected BasePage(WebDriver driver) {
        this.driver = Objects.requireNonNull(driver, "Driver must not be null");
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    protected WebElement waitForVisibility(By locator) {
        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));
    }

    protected void click(By locator) {
        wait.until(ExpectedConditions.elementToBeClickable(locator)).click();
    }

    protected void type(By locator, String text) {
        WebElement element = waitForVisibility(locator);
        element.clear();
        element.sendKeys(text);
    }

    // Template method: Enforces that every page subclass verifies its own page readiness
    public abstract boolean isLoaded();
}

```

4. Anti-Patterns

- ✗ **BaseTest extends BasePage** : Violates the "Is-A" contract. A test suite is NOT a web page. This anti-pattern binds test lifecycle directly to UI state, preventing multi-tab or API-to-UI hybrid testing.
- ✗ **Deep hierarchies (>2 levels)**: If `UserAdminPage` extends `AdminPage` which extends `AuthenticatedPage` which extends `BasePage`, debugging constructor failures or driver state becomes a nightmare.

2.3 Polymorphism

1. Theory & Core Mechanics

Polymorphism ("many forms") allows objects of different types to be treated through a unified interface.

1. **Compile-Time (Static) Polymorphism:** Method Overloading. The compiler determines which method signature to bind based on argument count, type, and order at compile time.
2. **Runtime (Dynamic) Polymorphism:** Method Overriding. The JVM determines which method implementation to execute at runtime based on the actual object instance on the heap, using dynamic method dispatch via the virtual method table (`vtable`).

2. Enterprise Relevance: Multi-Browser & Cross-Platform Execution

Polymorphism is the foundation of cross-browser execution in enterprise frameworks. The test code interacts purely with the `WebDriver` interface (`driver.get()` , `driver.findElement()`). At runtime, whether the underlying instance is `ChromeDriver` , `FirefoxDriver` , `EdgeDriver` , or `RemoteWebDriver` , the test code remains 100% identical.

3. Production-Ready Code: Polymorphic Driver Execution

```
package com.deloitte.sdet.driver;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.FirefoxOptions;

public final class BrowserDriverFactory {

    public enum BrowserType { CHROME, FIREFOX }

    public static WebDriver createDriver(BrowserType type, boolean headless) {
        // Upcasting: Concrete browser implementations are polymorphically assigned to WebDriver
        return switch (type) {
            case CHROME -> {
                ChromeOptions options = new ChromeOptions();
                if (headless) options.addArguments("--headless=new");
                yield new ChromeDriver(options);
            }
            case FIREFOX -> {
                FirefoxOptions options = new FirefoxOptions();
                if (headless) options.addArguments("-headless");
                yield new FirefoxDriver(options);
            }
        };
    }
}
```

2.4 Abstraction

1. Theory & Core Mechanics

Abstraction is the process of exposing **what** an operation does while completely concealing **how** it is accomplished.

- It hides operational complexity behind clean, minimal conceptual boundaries.
- Achieved in Java via **Abstract Classes** (partial abstraction with shared state and concrete code) and **Interfaces** (pure behavioral contracts).

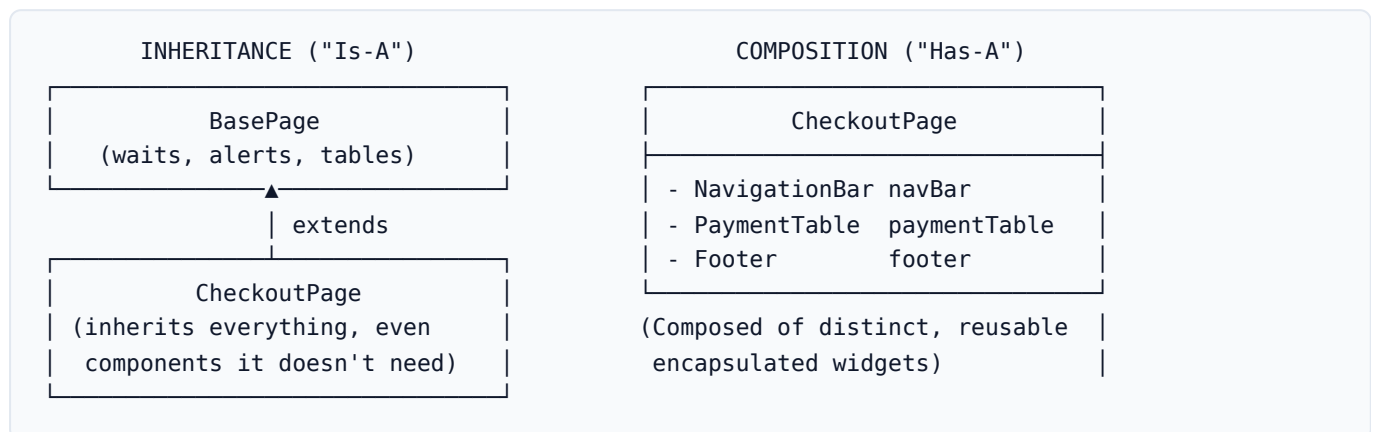
2. Enterprise Relevance: Storage & Environment Abstraction

In enterprise test frameworks, test data must be pulled dynamically from varied sources: local JSON files, remote AWS S3 buckets, or staging PostgreSQL databases. By coding tests against an abstract `TestDataProvider` interface, tests remain completely decoupled from the underlying storage mechanism:

```
public interface TestDataProvider {
    <T> T getTestData(String testKey, Class<T> modelClass);
}

// In local dev: JsonFileTestDataProvider implements TestDataProvider
// In CI/CD:      S3BucketTestDataProvider implements TestDataProvider
// In perf:       DatabaseTestDataProvider implements TestDataProvider
```

2.5 Composition vs. Inheritance



The Architectural Rule: Favor Composition Over Inheritance

- **Why Inheritance Breaks at Scale:** Inheritance tightly couples subclasses to superclass internals (white-box reuse). If a `NavigationBar` changes, extending it across 50 pages forces an inheritance tree rewrite.
- **Why Composition Wins:** Composition allows Page Objects to assemble independent, reusable Component Objects (black-box reuse). A `CheckoutPage` *has-a* `PaymentComponent`, *has-a* `NavigationBar`, and *has-a* `OrderSummaryComponent`.

Production-Ready Code: Component Object Model (COM)

```

package com.deloitte.sdet.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

// Component 1: Navigation Bar
public final class NavigationBarComponent {
    private final WebDriver driver;
    private final By searchBox = By.id("nav-search");

    public NavigationBarComponent(WebDriver driver) { this.driver = driver; }
    public void searchFor(String query) { driver.findElement(searchBox).sendKeys(query); }
}

// Component 2: Order Summary
public final class OrderSummaryComponent {
    private final WebDriver driver;
    private final By totalAmount = By.cssSelector(".order-total");

    public OrderSummaryComponent(WebDriver driver) { this.driver = driver; }
    public String getTotal() { return driver.findElement(totalAmount).getText(); }
}

// Composed Page: Assembles independent components via composition
public final class OrderConfirmationPage {
    private final NavigationBarComponent navBar;
    private final OrderSummaryComponent orderSummary;

    public OrderConfirmationPage(WebDriver driver) {
        this.navBar = new NavigationBarComponent(driver);
        this.orderSummary = new OrderSummaryComponent(driver);
    }

    public NavigationBarComponent nav() { return navBar; }
    public OrderSummaryComponent summary() { return orderSummary; }
}

```

2.6 Association, Aggregation, and Composition

Relationship	Type	Lifecycle Coupling	UML Multiplicity Example in SDET
Association	"Uses-A"	Independent lifecycles.	<code>TestClass</code> uses <code>WaitUtils</code> . <code>WaitUtils</code> exists independently of any test.
Aggregation	"Has-A" (Weak)	Independent lifecycles; child can exist without parent.	<code>TestSuite</code> aggregates <code>TestCase</code> objects. If the suite is garbage collected, test case definitions still exist.
Composition	"Has-A" (Strong)	Co-dependent lifecycle; child CANNOT exist without parent.	<code>PageObject</code> composes internal <code>PageHeader</code> . When <code>PageObject</code> is garbage collected, its <code>PageHeader</code> is destroyed with it.

2.7 Interface vs. Abstract Class

INTERFACE vs. ABSTRACT CLASS DECISION TREE	
Choose an INTERFACE when:	Choose an ABSTRACT CLASS when:
1. Defining a purely behavioral contract ("Can-Do")	1. Sharing code/state across closely related classes
2. Multiple classes with no relationship share capability (e.g., <code>TakesScreenshot`</code>)	2. Fields need non-static or private/protected visibility
3. You need multiple inheritance of type (implements A, B, C)	3. Enforcing constructor logic across all subclasses (e.g. mandatory <code>WebDriver`</code> non-null)

Detailed Comparison Matrix

Dimension	Interface	Abstract Class
State / Fields	Only <code>public static final</code> constants.	Can have instance variables (<code>private</code> , <code>protected</code>).
Constructors	Cannot have constructors.	Can have constructors invoked via <code>super()</code> .
Methods	<code>abstract</code> , <code>default</code> , <code>static</code> , <code>private</code> .	<code>abstract</code> , concrete with any access modifier.
Inheritance	A class can implement multiple interfaces .	A class can extend only one abstract class .

2.8 Default and Static Interface Methods (Java 8+)

1. Theory & Core Mechanics

Introduced in Java 8 to allow interface evolution without breaking backward compatibility:

- **default methods:** Provide concrete implementations within an interface. Implementing classes inherit the behavior or can override it.
- **static methods:** Utility methods belonging to the interface namespace. Must be invoked via `InterfaceName.method()` ; never inherited by implementing classes.

The Diamond Problem Resolution Rules

If class `C` implements interfaces `A` and `B` , both providing the exact same default method `void log()` :

1. **Rule 1 (Class always wins):** A superclass method declaration takes priority over any interface default method.
2. **Rule 2 (Sub-interface wins):** If `B` extends `A` , `B` 's default method takes priority.

3. **Rule 3 (Explicit Conflict Resolution):** If `A` and `B` are unrelated, the compiler fails with `class C inherits unrelated defaults for log() from types A and B`. Class `C` **must explicitly override** the method:

```
@Override
public void log() {
    A.super.log(); // Explicitly disambiguate and delegate to A's implementation
}
```

2.9 Method Overloading & Type Promotion Rules

1. Theory & Resolution Hierarchy

Method overloading is resolved at **compile time**. The compiler matches signatures according to this strict priority order:

1. **Exact Type Match**
2. **Primitive Widening** (`byte` `$to$` `short` `$to$` `int` `$to$` `long` `$to$` `float` `$to$` `double`)
3. **Autoboxing / Unboxing** (`int` `$to$` `Integer`)
4. **Varargs** (`int...`)

[!WARNING]

The compiler will NEVER perform Widening followed by Autoboxing (e.g., `int` will widen to `long` , but will NOT widen to `long` and then autobox to `Long`).

```
public class OverloadResolver {
    public static String resolve(long val)    { return "Widening to long"; }
    public static String resolve(Integer val) { return "Autoboxing to Integer"; }
    public static String resolve(int... vals) { return "Varargs"; }

    public static void main(String[] args) {
        int x = 10;
        // Widening beats Autoboxing: Outputs "Widening to long"
        System.out.println(resolve(x));
    }
}
```

2.10 Method Overriding & Covariant Return Types

1. The Rules of Overriding (R-A-M-T Rule)

1. **Return Type:** Must be identical or a **Covariant Subtype** (a more specific subclass of the parent return type).
2. **Access Modifier:** Cannot be more restrictive than the parent method (e.g., if parent is `protected` , override can be `protected` or `public` , but never `private`).
3. **Method Signature:** Exact method name and parameter types must match.

4. **Throws Clause:** Cannot throw new or broader **checked exceptions** than the parent method. It can throw fewer checked exceptions, subclasses of the parent checked exceptions, or any unchecked (`RuntimeException`).

Production Code: Covariant Return Types in Fluent Page Objects

```
public abstract class MasterPage {
    public abstract MasterPage refresh();
}

public final class InventoryPage extends MasterPage {
    // Covariant Return: Returns InventoryPage instead of generic MasterPage
    @Override
    public InventoryPage refresh() {
        driver.navigate().refresh();
        return this; // Allows fluent method chaining without downcasting in the test!
    }
}
```

2.11 Dynamic Method Dispatch (JVM vtable Internals)

1. How the JVM Executes Overridden Methods

When `driver.findElement(By.id("btn")).click()` is executed:

1. At compile time, the bytecode instruction generated is `invokevirtual` targeting the symbolic reference `WebElement.click()`.
2. At runtime, the JVM does NOT perform a slow string lookup of the method name.
3. Every loaded class in Metaspace has a **Virtual Method Table (vtable)**, which is an array of direct memory pointers to the actual executable machine code of each method.
4. If class `RemoteWebElement` overrides `click()`, its `vtable` slot for `click` points directly to `RemoteWebElement.click()`.
5. The CPU performs a fast, constant-time indirect array lookup: `objectRef -> KlassPointer -> vtable[slot_index] -> jump to machine code`.

2.12 Constructor Chaining (`this()` and `super()`)

1. Theory & Execution Sequence

When an object is instantiated, constructor chaining ensures that state is initialized from the top of the inheritance hierarchy downward:

1. Superclass static initializers (once on class load)
2. Subclass static initializers (once on class load)
3. Superclass instance initializers & field inline defaults
4. Superclass constructor body

5. Subclass instance initializers & field inline defaults
6. Subclass constructor body

- `this(...)` calls an overloaded constructor within the same class.
- `super(...)` calls a constructor in the immediate superclass.
- Either `this(...)` or `super(...)` must be the **very first statement** in a constructor.

2.13 `this` VS. `super`

Keyword	Definition	Primary Use Cases
<code>this</code>	Reference to the current executing object instance.	1. Disambiguating shadowed instance fields (<code>this.driver = driver</code>). 2. Constructor chaining (<code>this("defaultUser")</code>). 3. Returning current instance for Fluent APIs (<code>return this</code>).
<code>super</code>	Reference to the immediate superclass view of the object.	1. Invoking parent constructors (<code>super(driver)</code>). 2. Invoking overridden parent methods (<code>super.click(locator)</code>).

2.14 Access Modifiers

Access Modifier	Same Class	Same Package	Subclass (Diff)	World
<code>`private`</code>	YES	NO	NO	NO
<code>[default]</code>	YES	YES	NO	NO
<code>`protected`</code>	YES	YES	YES	NO
<code>`public`</code>	YES	YES	YES	YES

Enterprise Framework Best Practice

- **Locators & Internal State:** Strictly `private`.
- **BasePage Helper Utilities:** `protected` (accessible only to extending Page Objects, invisible to tests).
- **Public Business Workflows:** `public` (exposed to test cases).
- **Internal Helper Classes:** Package-private (default), keeping framework plumbing hidden inside its utility package.

2.15 `static` Mechanics

1. Theory & Memory Allocation

- Static variables belong to the **Class metadata** (stored in the Heap as part of the `java.lang.Class` instance), NOT to individual object instances.
- Loaded exactly once when the class is initialized by its `ClassLoader`.
- Static methods have no `this` reference; they cannot access instance fields or call instance methods directly.

2. The Catastrophic Anti-Pattern: `public static WebDriver driver`

```
// CATASTROPHIC ANTI-PATTERN FOR PARALLEL TESTING
public class BaseTest {
    public static WebDriver driver; // SHARED ACROSS ALL PARALLEL THREADS!
}
```

- **What happens in parallel execution:** Thread 1 launches Chrome and assigns it to `driver`. Thread 2 launches Firefox and overwrites the exact same `static driver` pointer. Thread 1 attempts to click a button, but interacts with Firefox or crashes with `NoSuchSessionException` because Thread 2 closed it.
- **The Senior Fix:** Encapsulate the driver inside a `ThreadLocal<WebDriver>` or inject it via dependency injection (TestNG `@Parameters` / PicoContainer).

2.16 `final` Mechanics

- **`final` Variable:** Cannot be reassigned once initialized. For reference types, the **pointer is immutable**, but the object's internal fields CAN still be mutated!
- **`final` Method:** Cannot be overridden by subclasses. Enables JIT inlining optimizations.
- **`final` Class:** Cannot be extended (e.g. `java.lang.String`, `java.lang.System`). Prevents malicious or erroneous inheritance.

2.17 SOLID Principles (Deep Theoretical Mechanics)

SOLID PRINCIPLES OVERVIEW	
[S] Single Responsibility	One class, one single reason to change
[O] Open/Closed Principle	Open for extension, closed for mod
[L] Liskov Substitution	Subtypes must be substitutable for base
[I] Interface Segregation	Small, role-specific client interfaces
[D] Dependency Inversion	Depend on abstractions, not concretions

2.18 SOLID Applied to Test Automation (5,000+ Test Suite Scale)

1. S — Single Responsibility Principle (SRP)

- **Violation:** The "God BasePage" anti-pattern. A single `BasePage` containing 1,500 lines handling Selenium clicks, REST Assured calls, database queries, and test assertions.

- **Senior Solution:** Split responsibilities:
 - `LoginPage` : UI DOM interactions only.
 - `UserApiClient` : Backend HTTP user provisioning only.
 - `UserDatabaseRepository` : Database SQL state verification only.
 - `TestClass` : Assertions and orchestration only.

2. O — Open / Closed Principle (OCP)

- **Violation:** A `DriverFactory` using an ever-growing `if-else` or `switch` statement to configure browser options. Adding a new browser requires modifying the stable factory class.
- **Senior Solution:** Strategy Pattern. Define a `DriverStrategy` interface. Adding `EdgeDriver` simply means creating a new `EdgeDriverStrategy` class without touching existing production code.

3. L — Liskov Substitution Principle (LSP)

- **Violation:** A `MobileAppPage` extends `BaseWebPage`, but throws `UnsupportedOperationException("Web hover not supported on mobile")` when `hover()` is called.
- **Senior Solution:** Segregate mobile and web capabilities into distinct behavioral abstractions. Any derived page must fulfill all behavioral contracts of its parent without throwing unexpected exceptions.

4. I — Interface Segregation Principle (ISP)

- **Violation:** A fat interface `AutomationActions` forcing implementing classes to implement 50 methods: `click()`, `doubleClick()`, `uploadFile()`, `dragAndDrop()`, `executeAsyncScript()`.
- **Senior Solution:** Break into fine-grained, cohesive interfaces: `Clickable`, `Scrollable`, `Uploadable`, `ScriptExecutable`.

5. D — Dependency Inversion Principle (DIP)

- **Violation:** `LoginPage` instantiates a concrete `new ChromeDriver()` inside its constructor. The page cannot be tested against Firefox, Edge, or a mock driver.
- **Senior Solution:** High-level modules (`LoginPage`) depend on abstractions (`WebDriver`), injected via constructor injection or a test context container.

2.19 High-Stakes Senior OOP Interview Questions & Whiteboard Scripts

Q1: "Why do you favor Composition over Inheritance when designing Page Object frameworks?"

"Inheritance binds classes at compile time through white-box reuse. In a test automation framework, extending a `BasePage` across hundreds of pages leads to the 'Fragile Base Class' problem: a change in the base class can unpredictably break unrelated pages, and child pages inherit methods they do not need.

Composition represents 'Has-A' relationships through black-box reuse. Modern web applications are composed of discrete, reusable components—such as Navigation Bars, Modals, and Data Tables. By modeling these as

independent Component Objects and composing them inside Page Objects, we achieve modular, highly reusable components that can be tested in isolation and updated in a single place when DOM structures change."

Q2: "Can you walk me through the exact SOLID violation when assertions are placed inside Page Objects?"

*"Placing assertions inside Page Objects violates the **Single Responsibility Principle (SRP)**.*

A Page Object has one single responsibility: encapsulating the structural representation and behavioral interactions of a web page. When you add assertions (e.g.,

Assert.assertTrue(welcomeMsg.isDisplayed())) inside a page method, the Page Object now has two responsibilities: interacting with the page AND validating business test logic.

*This creates two severe architectural drawbacks:

1. It destroys Page Object reusability: You cannot reuse that method in a negative test case or an exploratory test where the element is expected to be absent.
2. It obscures failure root-cause analysis in CI reports: The failure stack trace points to a Page Object line rather than the `@Test` class, misleading triage engineers into assuming a locator failure rather than an intentional assertion verification failure."*